SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security. (BL3, BL6)

Assessment Tool Weightage: 20%

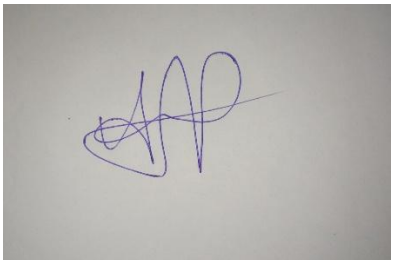
QUESTIONS: 5 **TOTAL MARKS: 25**

Industry Problem-Solving Task

Code of Conduct:

I Mohammed Arshad R/192521126(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Question 1: SSL/TLS Overhead and Web Server Performance Optimization

Given: payload = 2000 bytes/response; TLS record overhead = 25 bytes/response; handshake overhead = 3 KB (3000 bytes) per new session; 10,000 requests/sec; 1,000 new TLS handshakes/sec. (1 KB = 1000 bytes used throughout.)

(A) Total data transmitted per second, including TLS overhead

Per-response data = payload + TLS record overhead = 2000 + 25 = 2025 bytes

Response traffic = 10,000 × 2025 = 20,250,000 bytes/s

Handshake traffic = 1,000 × 3000 = 3,000,000 bytes/s

Total data transmitted per second = 20,250,000 + 3,000,000 = 23,250,000 bytes/s ≈ 23.25 MB/s

(B) Percentage overhead due to TLS encryption

Total TLS overhead = (10,000 × 25) + (1,000 × 3000) = 250,000 + 3,000,000 = 3,250,000 bytes/s

Base payload-only traffic = 10,000 × 2000 = 20,000,000 bytes/s

Percentage overhead = 3,250,000 / 20,000,000 × 100 = 16.25% (relative to raw payload)

(Equivalently, TLS overhead makes up 3,250,000 / 23,250,000 × 100 ≈ 13.98% of total bytes actually transmitted.)

(C) Bandwidth consumed due to handshake overhead per second

Handshake bandwidth = 1,000 handshakes/s × 3000 bytes = 3,000,000 bytes/s = 3 MB/s (24 Mbps)

(D) Trade-off analysis and optimization techniques

Only 1,000 of the 10,000 requests/sec (10%) trigger a full handshake, so 3 MB/s of the 23.25 MB/s total — about 13% — is pure handshake overhead rather than actual content. Full TLS handshakes are the most expensive part of the exchange: they add round-trip latency (extra network round trips before any data flows) and CPU cost (asymmetric-key operations for key exchange and certificate verification), so the security they provide (fresh authenticated key material per session) is paid for in both bandwidth and response latency.

- Session resumption (session IDs or session tickets): lets a returning client skip the full asymmetric handshake and reuse previously negotiated key material, cutting most repeat connections down to an abbreviated handshake — directly reducing how often the 3000-byte/1000-per-second handshake cost is paid.
- HTTP/2 multiplexing: allows many logical requests to share a single underlying TLS connection instead of opening a new connection (and therefore a new handshake) per request, amortizing the handshake cost across far more than one response.
- QUIC / HTTP/3: combines the transport and TLS 1.3 handshake into fewer round trips, and supports 0-RTT resumption where a returning client can send request data immediately without waiting for a full handshake to complete — directly attacking the latency side of the trade-off.

The trade-off is that faster resumption/0-RTT mechanisms slightly weaken per-session guarantees (0-RTT data is technically replayable by an attacker who captures it), so these optimizations should be applied carefully — e.g., avoiding 0-RTT for non-idempotent operations such as financial transactions — rather than as a blanket replacement for full handshakes.

Question 2: PGP-Based Email Encryption Load in Enterprise System

Given: 10 MB attachment/email (1 MB = 1,000,000 bytes); PGP overhead = 800 bytes/email; 5,000 emails/day.

(A) Total data transmitted per day, including overhead

Per-email size = 10,000,000 + 800 = 10,000,800 bytes

Total per day = 5,000 × 10,000,800 = 50,004,000,000 bytes ≈ 50.004 GB/day

(B) Percentage increase in storage due to PGP

Percentage increase = 800 / 10,000,000 × 100 = 0.008% per email

This is negligible because the fixed PGP overhead (key-encryption block + signature, 800 bytes) is minuscule compared to a 10 MB attachment — PGP's storage cost is completely dominated by the attachment size, not the cryptographic metadata.

(C) Computational overhead during encryption/decryption

PGP's hybrid design means the expensive asymmetric (RSA) operation only ever processes the small session key — its cost per email is small and constant. The dominant computational cost instead scales with the attachment volume actually being encrypted: 5,000 emails × 10 MB = 50,000 MB (50 GB) of data must pass through symmetric (AES) encryption, and typically through PGP's compression step (zlib or similar) as well, every single day. At scale, this compression pass can actually become the practical bottleneck rather than the AES encryption itself, since general-purpose compression is comparatively CPU-intensive, while modern AES benefits heavily from hardware acceleration (AES-NI). The RSA signature generation per email is a small, fixed, additional cost layered on top.

(D) Recommended optimization strategies

- Compression tuning: skip (or reduce) the compression step for attachments that are already compressed formats (JPEG, ZIP, MP4, etc.), since compressing already-compressed data wastes CPU cycles for essentially no size reduction — most modern PGP implementations already detect this, but it should be explicitly verified/enabled.
- Batching: queue and process emails in batches during off-peak periods where possible, smoothing CPU load rather than requiring peak capacity to be provisioned for worst-case simultaneous bursts.
- Hardware acceleration: use AES-NI-enabled hardware (standard on modern server CPUs) and, for very high volume, dedicated cryptographic accelerator cards or HSMs to offload bulk AES throughput from general-purpose CPU cycles.
- Parallel processing: since each email's encryption is independent of every other email's, distribute the 5,000/day workload across multiple worker processes/cores (or servers), which scales near-linearly with added hardware.

Question 3: IPSec Transport vs. Tunnel Mode Performance Comparison

Given: original packet = 1000 bytes payload + 20-byte header (1020 bytes base); Transport Mode overhead = +30 bytes; Tunnel Mode overhead = +50 bytes; traffic = 20,000 packets/sec.

(A) Total packet size for both modes

Mode	Base packet	Mode overhead	Total packet size
Transport Mode	1020 bytes	+30 bytes	1050 bytes
Tunnel Mode	1020 bytes	+50 bytes	1070 bytes

(B) Bandwidth usage for each mode per second

Transport Mode: 1050 × 20,000 = 21,000,000 bytes/s = 21 MB/s (168 Mbps)

Tunnel Mode: 1070 × 20,000 = 21,400,000 bytes/s = 21.4 MB/s (171.2 Mbps)

Tunnel Mode consumes 400,000 bytes/s (≈ 1.90%) more bandwidth than Transport Mode at this traffic volume.

(C) Efficiency and security comparison

Efficiency: Transport Mode is more bandwidth-efficient — it only encrypts/authenticates the payload and reuses the original IP header, costing 20 fewer overhead bytes per packet than Tunnel Mode (30 vs 50 bytes), which adds up to 400,000 bytes/s at this traffic volume.

Security: Transport Mode leaves the original source and destination IP addresses fully visible to anyone observing the network path, since only the payload is protected. Tunnel Mode encrypts the entire original packet (header included) and re-encapsulates it inside a new outer packet, so the real endpoints and internal addressing scheme are completely hidden from any observer between the two IPSec gateways — a meaningfully stronger security posture at the cost of the extra 20 bytes/packet overhead.

(D) Recommendation for internal vs. external communication

Use Transport Mode for internal communications within the company's already-trusted, segmented internal network: the endpoints don't need to be hidden from other internal systems, and the bandwidth savings (≈ 1.9% at this volume, more in absolute terms at higher throughput) are worth capturing since the traffic never leaves a controlled perimeter.

Use Tunnel Mode for external / site-to-site communications — e.g., branch-office-to-headquarters traffic crossing the public internet, or remote-access VPN connections — where the traffic traverses an untrusted network and both the data and the internal network topology (real source/destination addresses) need to be protected from observers on that untrusted path. The modest extra overhead (20 bytes/packet, ≈ 1.9% more bandwidth here) is a reasonable cost for this materially stronger security guarantee when crossing a network the organization does not control.

Question 4: S/MIME Certificate Management in a Large Organization

Given: 12,000 employees; certificate size = 2 KB each; annual renewal; old and new certificates both stored simultaneously for 1 month during renewal.

(A) Total storage required during the renewal period

Steady-state storage (one certificate per employee) = 12,000 × 2 KB = 24,000 KB = 24 MB

During the 1-month overlap, BOTH old and new certificates are retained per employee: 12,000 × 2 × 2 KB = 48,000 KB = 48 MB

This means certificate storage temporarily doubles from a steady-state 24 MB to 48 MB for the duration of the one-month renewal window — an additional 24 MB that must be provisioned for, even though it is only needed transiently.

(B) Certificate distribution overhead across the network

Distributing 12,000 new 2 KB certificates = 12,000 × 2 KB = 24,000 KB = 24 MB of certificate data that must be pushed out across the network during the renewal rollout.

Spread evenly across the roughly 30-day renewal window, this is only ≈ 0.8 MB/day of raw certificate data — trivial in absolute bandwidth terms. The real overhead at this scale is operational rather than bandwidth-related: 12,000 individual directory/LDAP update transactions, mail-client configuration pushes, user notifications, and corresponding CRL/OCSP updates referencing the newly issued certificates, all of which carry per-transaction overhead well beyond the raw 2 KB certificate payload itself.

(C) Challenges in certificate lifecycle management at scale

- Dual-trust window management: both old and new certificates must remain valid and correctly recognized by mail clients simultaneously for a month, without breaking the ability to send/receive encrypted mail during the transition.
- Distribution reliability: ensuring all 12,000 employees' mail clients are correctly updated with the new certificate, with no manual step being missed or delayed for any individual user.
- Timely revocation: promptly retiring/revoking old certificates once the overlap period ends, and keeping CRL/OCSP responders synchronized with 12,000 newly issued and 12,000 newly retired certificates.
- Historical access continuity: old private keys (or key escrow) must remain available to decrypt previously received email that was encrypted under the old certificate, even after that certificate is formally retired.
- Operational/helpdesk load: any employee who fails to update in time loses the ability to send or receive encrypted mail correctly, typically generating a spike in support requests concentrated around the renewal date.
- PKI infrastructure scalability: the CA's signing throughput and the directory/LDAP infrastructure must handle a large batch of renewals, particularly if many renewals are concentrated in a short window.

(D) Optimization strategies

- Automation: automate the full certificate lifecycle (issuance, renewal, distribution) using standard enrollment protocols (e.g., SCEP/EST) integrated with the organization's directory/MDM systems, removing manual per-employee steps entirely.
- Centralized PKI: operate a centralized, managed internal Certificate Authority (e.g., Active Directory Certificate Services or an equivalent enterprise PKI) so certificate issuance, distribution, and revocation follow one consistent, auditable policy and pipeline.
- Short-lived certificates: move toward shorter validity periods with fully automated renewal (mirroring the industry-wide shift seen in public TLS certificates) — this reduces the impact of any single compromised certificate and, if renewal is fully automated and seamless, can reduce reliance on long manual dual-storage overlap windows.
- Staggered rollout: renew certificates on a rolling schedule (e.g., by department or employee ID range) rather than all 12,000 at once, smoothing distribution load and helpdesk demand over time instead of concentrating it into a single spike.

Question 5: Email Encryption Latency in Cloud-Based Systems

Given: AES encryption = 0.2 ms/MB; RSA key exchange = 5 ms/email (fixed); attachment = 8 MB/email; 8,000 emails/hour.

(A) Total encryption time per email

AES time = 0.2 ms/MB × 8 MB = 1.6 ms

RSA key-exchange time = 5 ms (fixed per email, independent of attachment size)

Total encryption time per email = 1.6 + 5 = 6.6 ms

(B) Total processing time per hour

Total = 8,000 emails × 6.6 ms = 52,800 ms = 52.8 seconds of cryptographic processing per hour

Broken down: AES processing totals 8,000 × 1.6 ms = 12,800 ms (12.8 s); RSA key exchange totals 8,000 × 5 ms = 40,000 ms (40 s).

(C) Performance bottleneck: symmetric vs. asymmetric operations

RSA key exchange is the dominant bottleneck: its 40 seconds/hour accounts for 75.8% of the total 52.8 seconds/hour of processing time (40 / 52.8 × 100), even though AES is the operation actually handling all 8 MB of attachment data per email. This is the classic asymmetric-vs-symmetric cost asymmetry: AES's simple, hardware-friendly operations scale efficiently with data volume, while RSA's large modular-exponentiation operations carry a comparatively heavy fixed cost per invocation regardless of how little data (just a small session key) they are actually protecting. Because RSA runs once per email regardless of attachment size, it is the per-email fixed cost that dominates total processing time at this volume, not the bulk data encryption.

(D) Recommended optimization strategies

- ECC adoption: replace RSA key exchange with Elliptic Curve Cryptography (e.g., ECDH/ECIES). ECC delivers equivalent security with much smaller keys and substantially faster operations than RSA, directly targeting the 40-second/hour bottleneck identified above.
- Parallel processing: since each email's encryption is independent, distribute the 8,000/hour workload across multiple CPU cores, worker processes, or servers — wall-clock throughput scales close to linearly with added parallelism.
- Hardware acceleration: leverage AES-NI (already fast for the symmetric portion) and, more importantly given the bottleneck, hardware-accelerated modular arithmetic (HSMs or crypto co-processors) to reduce the per-operation RSA/ECC cost.
- Reduce redundant asymmetric operations where safe to do so: for high-volume, policy-compliant scenarios, investigate precomputation or optimized exponentiation techniques for frequently-used recipient public keys, while still generating a fresh session key per email to preserve per-message security.